TP04 Report

Docker Images, Containers, Networking and Docker Compose

Big Data Practical Work

NourEddine Ben Attous

University of El Oued

November 4, 2026

Contents

Introduction **2**

1 Task 1 – Installation and Verification **3**
1.1 Definitions . . . 3
1.2 Commands . . . 3
1.3 Results . . . 3

2 Task 2 – Building and Publishing an Image **5**
2.1 Definitions . . . 5
2.2 Building the image . . . 5
2.3 Publishing and downloading the image . . . 6
2.4 Docker Hub results . . . 6

3 Task 3 – Containers and Networking **8**
3.1 Definitions . . . 8
3.2 Creating containers . . . 8
3.3 Observed results . . . 9
3.4 Creating and inspecting the network . . . 9

4 Task 4 – Docker Compose and CSV Processing **12**
4.1 Definition of Docker Compose . . . 12
4.2 What problem does Docker Compose solve? . . . 12
4.3 Running the application . . . 12
4.4 Difference between docker run and docker compose up . . . 13
4.5 CSV processing results . . . 13
4.6 Adding a database service . . . 13

5 Task 5 – Dockerfile and Theoretical Questions **15**
5.1 What is Docker and why is it useful in data science? . . . 15
5.2 What is the difference between an image and a container? . . . 15
5.3 Dockerfile instructions . . . 15
5.4 Docker Compose questions . . . 16
5.4.1 What problem does Docker Compose solve? . . . 16
5.4.2 What is the difference between docker run and docker compose up? 16
5.4.3 How can a database be added to Compose? . . . 16

6 Conclusion **17**

A Screenshot Checklist **18**

Introduction

The objective of this practical work is to understand the Docker ecosystem and its use in application deployment and data processing.

The practical work covers five tasks:

1. Installing and verifying Docker.
2. Building and publishing a Docker image.
3. Creating containers and connecting them to a network.
4. Using Docker Compose to execute a CSV-processing application.
5. Documenting the work and explaining the main Docker concepts.

The main Docker image created during this practical work is:

https://hub.docker.com/r/ennori/tp04-data-analysis

The image is published with tag `1.0`.

Chapter 1

Task 1 – Installation and Verification

1.1 Definitions

Docker A platform for building, distributing, and running applications in containers.

Docker Engine The service responsible for managing images, containers, networks, and volumes.

Docker CLI The command-line interface used to communicate with Docker Engine.

Docker Hub An online registry used to store and share Docker images.

1.2 Commands

The following commands were executed in PowerShell to verify the Docker installation and inspect the local environment.

Listing 1.1: Checking Docker installation

```
docker version
docker info
docker images
```

1.3 Results

The command `docker version` displayed information about the Docker Client and Server. This confirmed that the Docker CLI could communicate with the Docker Engine.

The command `docker info` displayed information about the Docker environment, including container and image counts.

The command `docker images` listed the images available locally. The image `ennori/tp04-data-ana` was present.

The Docker Hub account was authenticated using:

```
docker login
```

This authentication allows the user to perform operations such as publishing images to Docker Hub.

```
PS D:\big-data\tp4> docker version
Client:
 Version:           29.9.2
 API version:       1.56
 Go version:        go1.26.10
 Git commit:        a82e528
 Built:             Tue Oct 20 19:44:07 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.97.0 (244584)
 Engine:
  Version:          29.9.2
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.10
  Git commit:       0367cbe
  Built:            Tue Oct 20 18:45:02 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.4.2
  GitCommit:        b7a25261e029d68978bbf290df0a62fc819b3c51
 runc:
  Version:          1.5.2
  GitCommit:        v1.5.2-0-g29dd3dc2
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

Figure 1.1: Docker Client and Docker Engine version verification.

```
PS D:\big-data\tp4> docker images

IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
ennori/ennori-nginx-demo:1.0         650d8c800438       93.6MB         26.3MB
ennori/tp04-data-analysis:1.0        ebc688a2e9bb        400MB         90.7MB    U
hello-world:latest                   11a3eb1597e8       25.9kB         9.49kB    U
tp04-data-analysis:1.0               48ee34d4ae06        400MB         90.7MB    U
PS D:\big-data\tp4> docker login
Authenticating with existing credentials... [Username: ennori]

i Info → To login with a different account, run 'docker logout' followed by 'docker login'


Login did not succeed, error: Error response from daemon: Get "https://registry-1.docker.io/v2/": net/http: request canceled while waiting for connection (Client.Timeout exceeded while awaiting headers)

USING WEB-BASED LOGIN

i Info → To sign in with credentials on the command line, use 'docker login -u <username>'


Your one-time device confirmation code is: MZQD-KTTQ
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser…

Login Succeeded
```

Figure 1.2: Local Docker images and successful Docker Hub authentication.

Chapter 2

Task 2 – Building and Publishing an Image

2.1 Definitions

Dockerfile A text file containing instructions used to build a Docker image.

Image A packaged template containing application files, dependencies, and default execution settings.

Tag A label used to identify an image version, such as `1.0`.

Registry A service used to store and distribute Docker images.

2.2 Building the image

The image was built and tagged as `ennori/tp04-data-analysis:1.0`.

Listing 2.1: Building the Docker image

```
docker build -t ennori/tp04-data-analysis:1.0 .
```

Explanation:

- `docker build` builds an image.
- `-t` assigns a name and tag to the image.
- `ennori` is the Docker Hub username.
- `tp04-data-analysis` is the image repository.
- `1.0` is the image tag.
- `.` specifies the current directory as the build context.

The local image was checked using:

```
docker images
```

2.3 Publishing and downloading the image

The image was published to Docker Hub using `docker push` and retrieved using `docker pull`.

Listing 2.2: Publishing and retrieving the image

```
docker login
docker push ennori/tp04-data-analysis:1.0
docker pull ennori/tp04-data-analysis:1.0
```

2.4 Docker Hub results

The Docker Hub tag page displays the following information:

Property	Observed value
Repository	`ennori/tp04-data-analysis`
Tag	`1.0`
Platform	`linux/amd64`
Compressed size	`86.46 MB`
Index digest	`sha256:ebc688a2e9bbc05febad06a8ef8b916be817b3f00706865c`
Manifest digest	`sha256:12535d3464581b884b9128587c0e194264ccbd02e51355f8`

Table 2.1: Docker Hub image details.

The image layers displayed on Docker Hub indicate that the image uses Python 3.11, sets the working directory to `/app`, copies `requirements.txt` and `app.py`, installs dependencies, and specifies `python app.py` as the default command.

Repository URL:

https://hub.docker.com/r/ennori/tp04-data-analysis

Tag details:

https://hub.docker.com/layers/ennori/tp04-data-analysis/1.0/images/sha256-12535d3464581b884b9128587c0e194264ccbd02e51355f86c25d0da7b4d7162

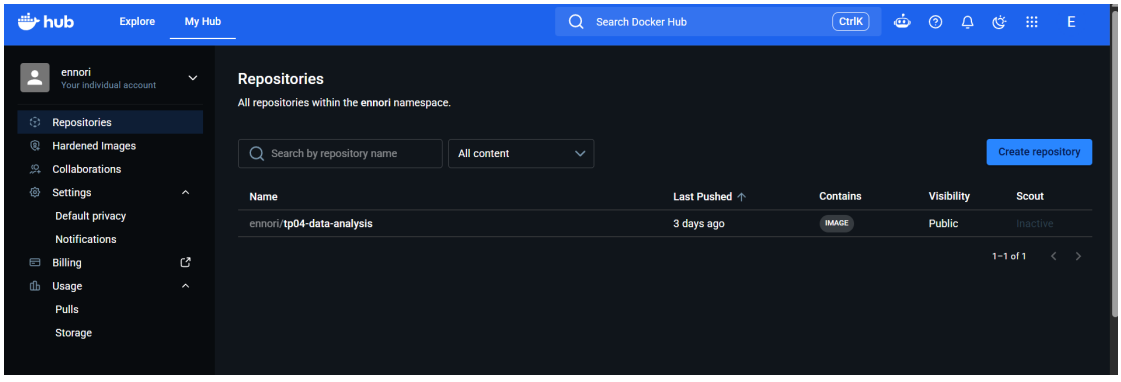


Figure 2.1: Docker Hub repository showing the published image.

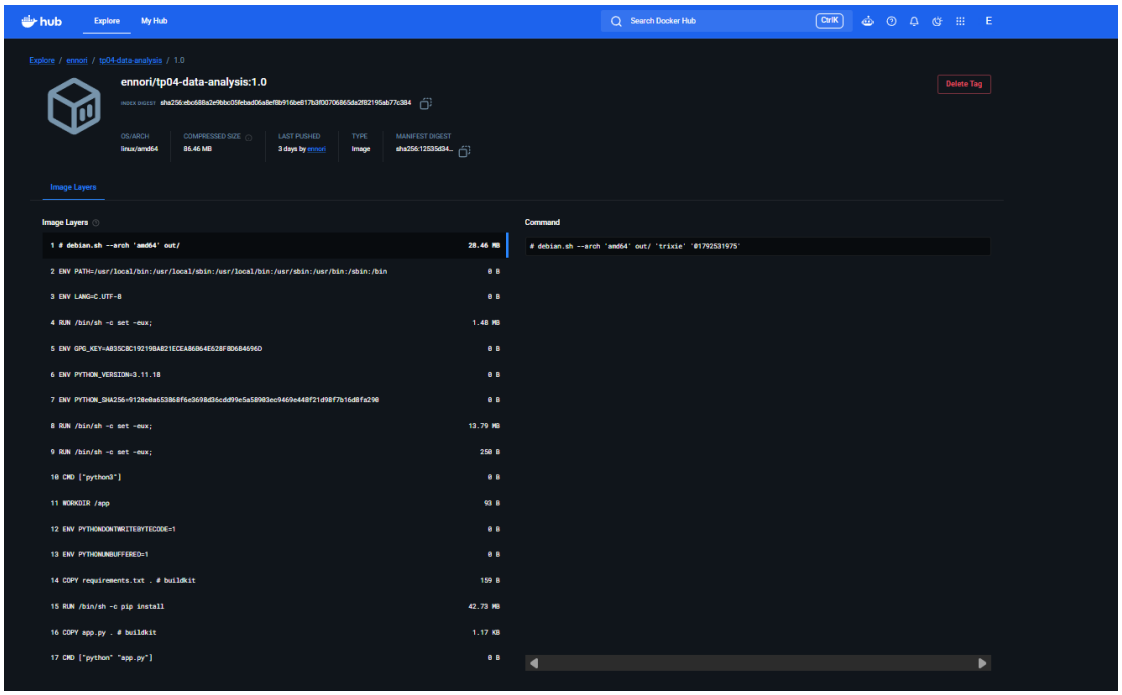


Figure 2.2: Docker Hub tag details, image layers, platform, and size.

```
PS D:\big-data\tp4> docker build -t ennori/tp04-data-analysis:1.0 .
[+] Building 37.5s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 246B
 => [internal] load metadata for docker.io/library/python:3.11-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 117B
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:673719e071ecf27434fd2635bbd8d00f49f4fdd81f243143438a42ced8bedf60
 => => resolve docker.io/library/python:3.11-slim@sha256:673719e071ecf27434fd2635bbd8d00f49f4fdd81f243143438a42ced8bedf60
 => => sha256:361db81715472ecba5e656c314c299bd94530902048315959 6a8d123c2b1d580 251B / 251B
 => => sha256:24ff04c921070df4f4dc987278926d5dabde59d57b55ae3665439331f79af932 14.46MB / 14.46MB
 => => sha256:ee5aaeba9945927662bea2702989dfd74d9273796415b896363e6a9c44d9a8ce 1.29MB / 1.29MB
 => => sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5 29.86MB / 29.86MB
 => => extracting sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5
 => => extracting sha256:ee5aaeba9945927662bea2702989dfd74d9273796415b896363e6a9c44d9a8ce
 => => extracting sha256:24ff04c921070df4f4dc987278926d5dabde59d57b55ae3665439331f79af932
 => => extracting sha256:361db81715472ecba5e656c314c299bd94530902048315959 6a8d123c2b1d580
 => [internal] load build context
 => => transferring context: 63B
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:116af938f4f6f2f885c90a1d8e1cfa85f12d1e5b814f407afae2a187f19a109f
 => => exporting config sha256:875439a20b3a678056a21d79c2b638624c920adf0cd17b0fe7264a9644622b43
 => => exporting attestation manifest sha256:2bd7e9b85852ef4f822b8fc362da4ea3f340082f400bea8e48b0659e5ed78d9c
 => => exporting manifest list sha256:06eb38b0e813ec92ccbe3c5d7a150eca8d3c5d3437c697a438a9062f865d96c0
 => => naming to docker.io/ennori/tp04-data-analysis:1.0
 => => unpacking to docker.io/ennori/tp04-data-analysis:1.0

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/tq4450rbvu1mscemxice2b63f
PS D:\big-data\tp4> docker push ennori/tp04-data-analysis:1.0
The push refers to repository [docker.io/ennori/tp04-data-analysis]
e0a0f980d9f0: Pushed
6fc242d221a6: Pushed
44136fa355b3: Already exists
ee5aaeba9945: Pushed
24ff04c92107: Pushed
361db8171547: Pushed
ccc9cd90bffa: Pushed
359d0b70f2cc: Pushed
ecd346f83e2d: Pushed
635bf1e136e4: Pushed
1.0: digest: sha256:06eb38b0e813ec92ccbe3c5d7a150eca8d3c5d3437c697a438a9062f865d96c0 size: 856
PS D:\big-data\tp4> docker pull ennori/tp04-data-analysis:1.0
1.0: Pulling from ennori/tp04-data-analysis
Digest: sha256:06eb38b0e813ec92ccbe3c5d7a150eca8d3c5d3437c697a438a9062f865d96c0
Status: Image is up to date for ennori/tp04-data-analysis:1.0
docker.io/ennori/tp04-data-analysis:1.0
```

Figure 2.3: PowerShell output showing build, push, and pull commands.

Chapter 3

Task 3 – Containers and Networking

3.1 Definitions

Container An isolated runtime instance created from a Docker image. A container can be running or stopped.

Docker Network A virtual network that allows connected containers to communicate according to its configuration.

Container Name A human-readable identifier used to manage a container.

3.2 Creating containers

Three containers were created:

- `tp04-node1`
- `tp04-node2`
- `tp04-node3`

They used the image `ennori/tp04-data-analysis:1.0`.
The following commands were used to inspect containers:

Listing 3.1: Displaying running containers

```
docker ps
```

Listing 3.2: Displaying all containers

```
docker ps -a
```

The command `docker ps` displays running containers by default. The command `docker ps -a` also displays stopped containers.

3.3 Observed results

The following required containers appeared in the actual PowerShell output:

Listing 3.3: Observed containers and statuses

```
CONTAINER ID   IMAGE                                COMMAND
e0c1ff3399fb   ennori/tp04-data-analysis:1.0   "python -c 'import t..."
d67a76150b4e   ennori/tp04-data-analysis:1.0   "python -c 'import t..."
626697c1fe39   ennori/tp04-data-analysis:1.0   "python -c 'import t..."

STATUS                       NAMES
Exited (0) 16 hours ago      tp04-node3
Exited (0) 16 hours ago      tp04-node2
Exited (0) 16 hours ago      tp04-node1
```

All three required containers exist. Their status is `Exited (0)`, which means that their main processes finished and returned exit code zero. This does not mean that they are currently running.

The containers were used to demonstrate container creation and network configuration. They were not necessarily three independent CSV-processing workers.

3.4 Creating and inspecting the network

A Docker network named `tp04-network` was created and the three containers were connected to it.

Listing 3.4: Listing Docker networks

```
docker network ls
```

Listing 3.5: Inspecting the Docker network

```
docker network inspect tp04-network
```

The inspection output can be used to verify which containers are connected to the network.

Being attached to the same network enables communication, but proving application-level communication requires an additional test, such as a successful request to a service running inside another container.

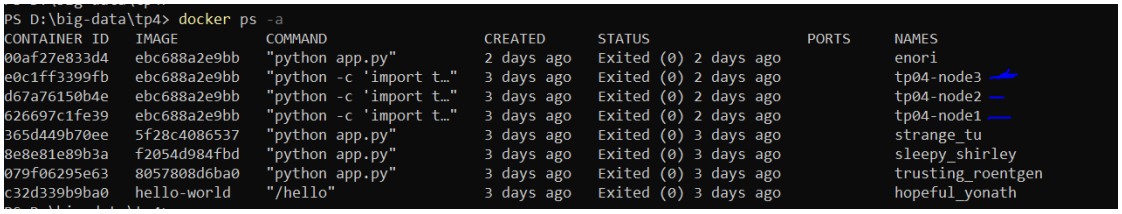


Figure 3.1: PowerShell output showing the three required containers.



```
PS C:\Users\msi\Desktop\big-data\tp4> docker network inspect tp04-network
[
    {
        "Name": "tp04-network",
        "Id": "5f27dae7c6c3acb799d49cd2aa1382a42448aa802552cd6617a227b9cd8a0096",
        "Created": "2026-10-27T15:03:24.282260771Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {
            "626697c1fe393ef152012ba27899066029264bf76f3dfc4b90061a6df4665937": {
                "Name": "tp04-node1",
                "EndpointID": "9de84b97f270c81a11e28110c132ad573135f25a1f9bd0cfd178a4fc845cb99a",
                "MacAddress": "22:5d:c6:54:e1:de",
                "IPv4Address": "172.18.0.2/16",
                "IPv6Address": ""
            },
            "d67a76150b4e7e86325035db3e98e050c7e76facfd2ad55afb07de357b240b9c": {
                "Name": "tp04-node2",
                "EndpointID": "844d39b99c0d83afba0598cb34de21a8b439533e60d2ad6dfb47d04b701c68a6",
                "MacAddress": "02:53:38:af:be:90",
                "IPv4Address": "172.18.0.3/16",
                "IPv6Address": ""
            },
            "e0c1ff3399fb6d47c12af5a4785af8d3b9b8c630a1536f21fe6b571977feffda": {
                "Name": "tp04-node3",
```


Figure 3.2: Docker network inspection for tp04-network.

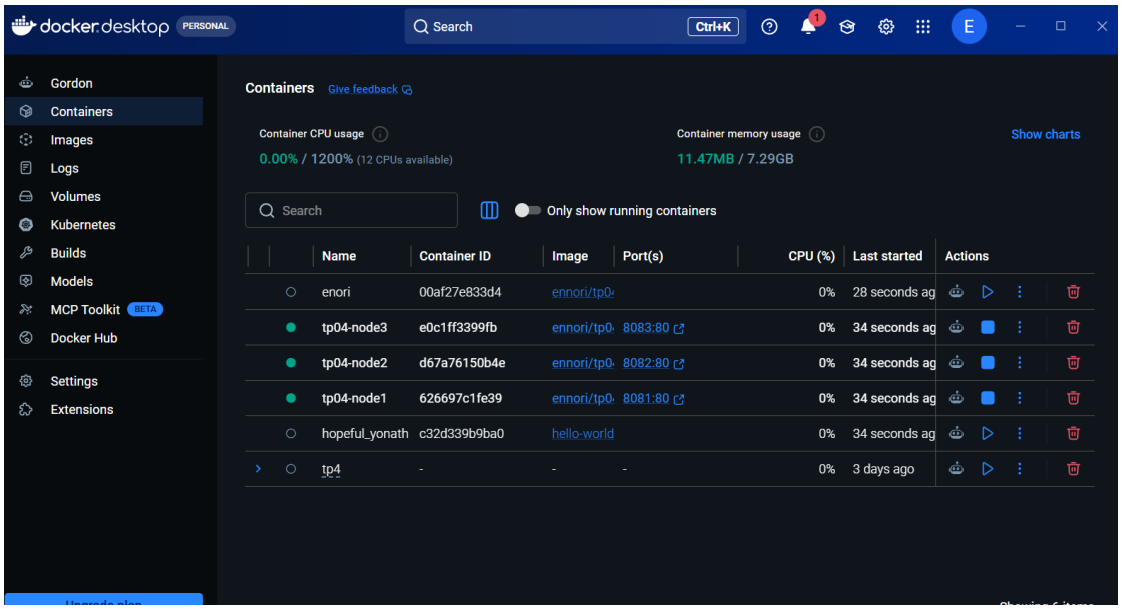


Figure 3.3: Docker Desktop Containers view.

Chapter 4

Task 4 – Docker Compose and CSV Processing

4.1 Definition of Docker Compose

Docker Compose is a tool for defining and managing application services through a YAML file, usually named `compose.yaml` or `docker-compose.yml`.

It allows service configuration to be stored in one place and makes it easier to start and manage an application composed of one or more services.

4.2 What problem does Docker Compose solve?

Without Compose, each container may need to be configured and started using separate commands. Compose centralizes build settings, environment variables, networks, volumes, and service dependencies.

It makes project execution more reproducible and simplifies the management of multi-service applications.

4.3 Running the application

The project was started with:

Listing 4.1: Building and starting Compose services

```
docker compose up --build
```

The `up` command creates and starts services defined in the Compose file. The `-build` option requests that images for services with a build configuration be built before the services start.

After execution, the services were inspected using:

Listing 4.2: Inspecting Compose containers

```
docker compose ps -a
```

The environment was cleaned up using:

Listing 4.3: Cleaning up Compose resources

```
docker compose down
```

4.4 Difference between docker run and docker compose up

- `docker run` creates and starts a container from an image using the supplied options.
- `docker compose up` starts the services defined in a Compose YAML file.

Compose is useful when an application consists of multiple services with shared networks, volumes, environment variables, or dependencies. The `docker run` command can also configure networks and volumes, but it does not manage a project from a Compose file.

4.5 CSV processing results

The application processed a large CSV file and completed with exit code `0`. The recorded output indicated approximately **233,055,326 rows processed**.

Exit code zero indicates that the program reported successful completion. However, it does not by itself guarantee that the CSV was interpreted correctly. The header configuration, column names, and analysis results should also be checked against the dataset format.

4.6 Adding a database service

A database can be added as another service under the `services` section of the Compose file.

The following is an illustrative example. It is not a claim that PostgreSQL was part of the executed project.

Listing 4.4: Example Compose file with PostgreSQL

```
services:
  app:
    build: .
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: analytics
      POSTGRES_USER: analyst
      POSTGRES_PASSWORD: change_me
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

```
volumes:
  postgres_data:
```

In this example:

- `app` is the application service.
- `db` is the PostgreSQL database service.
- `environment` defines database configuration.
- `volumes` provides persistent database storage.
- `depends_on` defines startup ordering, but does not guarantee that the database is ready to accept connections.

In real projects, sensitive passwords should not be committed directly to a public repository.

```
tp04-analysis  | PROCESSING COMPLETED
tp04-analysis  | Total rows: 233,055,326
tp04-analysis  | Total columns: 4
tp04-analysis  |
tp04-analysis  | COLUMN DATA TYPES
tp04-analysis  | B00ZGTMHVU         object
tp04-analysis  | A1HGH7NF01K3W7     object
tp04-analysis  | 5.0               float64
tp04-analysis  | 1451001600          int64
tp04-analysis  |
tp04-analysis  | MISSING VALUES (ALL ROWS)
tp04-analysis  | B00ZGTMHVU        0
tp04-analysis  | A1HGH7NF01K3W7    0
tp04-analysis  | 5.0               0
tp04-analysis  | 1451001600        0
tp04-analysis  |
tp04-analysis  | NUMERIC SUMMARY (ALL ROWS)
tp04-analysis  |                       mean          min           max  non_null_count
tp04-analysis  | 5.0          4.232190e+00          0.0  5.000000e+00       233055326
tp04-analysis  | 1451001600   1.428971e+09  832550400.0  1.538870e+09       233055326
tp04-analysis  |
tp04-analysis  | All CSV rows have been processed.
tp04-analysis exited with code 0
```

Figure 4.1: Docker Compose execution and CSV-processing output.

```
PS D:\big-data\tp4> docker compose ps -a
NAME           IMAGE                     COMMAND           SERVICE        CREATED         STATUS                          PORTS
tp04-analysis  tp04-data-analysis:1.0    "python app.py"   data-analysis  6 minutes ago   Exited (0) About a minute ago
```

Figure 4.2: Compose container status after execution.

```
PS D:\big-data\tp4> docker compose down
[+] down 2/2
 ✔Container tp04-analysis Removed
 ✔Network tp4_default     Removed
```

Figure 4.3: PowerShell output after docker compose down.

Chapter 5

Task 5 – Dockerfile and Theoretical Questions

5.1 What is Docker and why is it useful in data science?

Docker is a platform that packages an application and its dependencies into an image and runs it inside a container.

In data science, projects often depend on specific versions of Python and external libraries. Docker helps reproduce the same runtime environment on different machines, reduces dependency conflicts, and simplifies sharing the project.

5.2 What is the difference between an image and a container?

An image is a template containing application files, dependencies, and execution settings. A container is a runtime instance created from that image.

Multiple containers can be created from one image. A container can stop while the image remains available.

5.3 Dockerfile instructions

`FROM`	Selects the base image on which the new image is built. Example: `FROM python:3.11-slim`.
`WORKDIR`	Sets the working directory for subsequent instructions. Example: `WORKDIR /app`.
`COPY`	Copies files from the build context into the image. Example: `COPY requirements.txt .`.
`RUN`	Executes a command during image building, often to install dependencies. Example: `RUN pip install -r requirements.txt`.
`CMD`	Specifies the default command or arguments to execute when a container starts. Example: `CMD ["python", "app.py"]`.

The main distinction is that `RUN` executes during image building, while `CMD` defines the default process to run when a container starts.

5.4 Docker Compose questions

5.4.1 What problem does Docker Compose solve?

It centralizes the configuration of application services and allows them to be started and managed as one project using a YAML file.

5.4.2 What is the difference between docker run and docker compose up?

`docker run` starts an individual container using command-line options. `docker compose up` starts the services defined in the Compose configuration and manages the project resources required by those services.

5.4.3 How can a database be added to Compose?

A database is added as a service under `services`. Its image, environment variables, volumes, and networking configuration can be defined in the Compose file. The application must use the database service name as its network hostname and should wait until the database is ready.

Chapter 6

Conclusion

This practical work covered the main Docker workflow: verifying the installation, building an image, publishing and downloading it from Docker Hub, creating containers, connecting them to a Docker network, and running a CSV processing task through Docker Compose.

The published image is available at:

https://hub.docker.com/r/ennori/tp04-data-analysis

The Docker Hub tag page confirms the `1.0` image, the `linux/amd64` platform, and a compressed size of approximately `86.46 MB`.

The local PowerShell output confirms that `tp04-node1`, `tp04-node2`, and `tp04-node3` exist and exited with code `0`. The Compose execution also completed with exit code zero and reported processing approximately 233 million CSV rows.

Before final submission, the actual screenshots should be inserted and the CSV header handling should be checked to ensure that successful execution also corresponds to correct data interpretation.

Appendix A

Screenshot Checklist

Place the real screenshots in a folder named `screenshots/` beside this LaTeX file.

1. `01-docker-version-info.png`
2. `02-dockerhub-repository.png`
3. `03-dockerhub-tag-layers.png`
4. `04-docker-build-push-pull.png`
5. `05-containers-docker-ps-a.png`
6. `06-docker-network-inspect.png`
7. `07-docker-desktop-containers.png`
8. `08-compose-up-analysis-output.png`
9. `09-compose-ps-a.png`
10. `10-compose-down.png`

The report displays placeholders when a screenshot is missing. Replace all placeholders with real screenshots before submitting the final report.